Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

Synthetic Control and Experimental Design in One Validated Interface: The `mlsynth` Package for Python

Jared Greathouse
Georgia State University

Abstract

Synthetic control methods are established tools for estimating treatment effects in economics, policy, and marketing science. The literature now has dozens of variants for observational work, alongside a newer family of experimental-design methods. Each new synthetic-control method usually comes as a separate package with its own data conventions, result format, and inference. I introduce `mlsynth`, a Python library that collects much of the modern synthetic-control literature behind a common interface. The library spans both estimation and experimental design, a combination the surrounding ecosystem lacks. Every estimator is validated against its originating paper or a reference implementation, and these checks are maintained as a durable benchmark suite. I illustrate `mlsynth` with two observational applications and one experimental-design application.

Keywords: synthetic control, causal inference, panel data, experimental design, Python.

1. Introduction

A recurring problem in econometrics, marketing measurement, and policy analysis is estimating what would have happened to one unit (a state, a country, a market, a product line) absent some intervention. Randomized experiments are usually impossible at this aggregation: there is one California, and it either passed an anti-smoking proposition or it did not. The synthetic control method (Abadie and Gardeazabal 2003; Abadie, Diamond, and Hainmueller 2010) answers this by choosing a weighted average of untreated units, a *synthetic* California assembled from other states, with weights picked so the combination tracks the treated unit before the intervention. Carried past it, where the donors stay untreated, that same weighted average is the imputed counterfactual. It is now among the most widely used tools in program evaluation (Abadie 2021), and its original software, the `Synth` package (Abadie, Diamond, and Hainmueller 2011), appeared in this journal.

The simplex-weighted original sits alongside robust low-rank variants (Amjad, Shah, and Shen 2018), regularized variants (Abadie and L'Hour 2021; Ben-Michael, Feller, and Rothstein 2021), factor-model and interactive-fixed-effects approaches (Bai 2009; Xu 2017), matrix completion (Athey, Bayati, Doudchenko, Imbens, and Khosravi 2021), the panel-data approach of Hsiao, Ching, and Wan (2012) and its forward selection variant by Shi and Huang (2023), proximal and surrogate methods (Park and Tchetgen 2025; Shi, Li, Yu, Miao, Kuchibhotla, Hu, and Tchetgen 2026; Liu, Tchetgen Tchetgen, and Varjão 2024), and the synthetic-difference-in-differences estimator (Arkhangelsky, Athey, Hirshberg, Imbens, and Wager 2021).

Synthetic control also runs before an intervention, not only after it. Used for *experimental design*, the same weighting logic chooses which units to treat, and which to hold back as controls (Abadie and Zhao 2026; Doudchenko, Khosravi, Pouget-Abadie, Lahaie, Lubin, Mirrokni, Spiess, and Imbens 2021), so that a credible counterfactual exists once the experiment ends. Industry moved the same way: Meta's `GeoLift` (Meta Open Source 2024) brought the design-first idea to geographic marketing experiments, building valid geo-experiments on the augmented synthetic control of Ben-Michael *et al.* (2021).

This proliferation carries costs. Each new variant typically ships as replication code attached to its paper rather than an installable package (Liu *et al.* 2024; Park and Tchetgen 2025; Amjad *et al.* 2018; Malo, Eskelinen, Zhou, and Kuosmanen 2024); by *packaged* I mean installable from a standard channel (GitHub, `pip`, CRAN, Stata's SSC) and called as-is, without reassembling an author's replication archive by hand. What does ship scatters across languages even within one lineage: the doubly-robust proximal estimator of Qiu, Shi, Miao, Dobriban, and Tchetgen Tchetgen (2024) is in R and its surrogate sibling (Liu *et al.* 2024) in Python; the panel-data regression-control approach is split between the Stata command `rcm` (Yan and Chen 2022) and the R package `pampe` (Vega-Bayo 2015). Each package brings its own data format, argument names, notion of a result, and inference, so comparing two estimators on one dataset can mean reshaping the data twice, learning two APIs, and reconciling two result formats by hand.

Worse, the gap is uneven: the advances that would help analysts most often have no packaged implementation at all. The Synthetic Interventions estimator of Agarwal, Shah, and Shen (2026), which asks how a unit treated with one policy would have fared under another (Sevigny, Greathouse, and Medhin 2023), the refined leave-two-out placebo test for small donor pools of Lei and Sudijono (2025), and the predictor-selection method of i Bastida (2022) all ship, if at all, only as paper-replication scripts. Well-maintained packages do exist (Robbins and Davenport 2021; Shi and Huang 2023; Cattaneo, Feng, Palomba, and Titiunik 2025), but much of the new literature is not software a data scientist can use off the shelf.

All are capable tools, and `mlsynth` critiques none of them. The obstacle is not any one of them but their dispersion: they span at least two languages, with no shared data format, no shared notion of a result, and no way to run one against another on the same panel without redoing the work each time.

This paper introduces `mlsynth`, a Python (Harris, Millman, van der Walt *et al.* 2020) library that addresses this fragmentation. It ships 58 estimators[1] behind one data-preparation routine, one validated configuration-and-fit interface, and one standardized result. Most of these

[1] Counted as the public classes that implement the `fit` contract; several are dispatchers that host a family of related methods behind one class, so the number of distinct methods a user can run is larger.

estimators share the same recipe, as Section 2 makes precise: a treated-unit vector and a donor matrix, split into pre- and post-treatment periods, enter an optimization that returns weights, a counterfactual, and a gap. A single interface therefore makes them directly comparable on identical inputs. A researcher can fit a `Synth`-style simplex control, an `rcm`-style panel-data regression, and a `synthdid`-style difference-in-differences design on one data frame, compare them, and attach prediction-interval or conformal inference to each.

`mlsynth` aims to make this catalog accessible, free, and simple to call. The paper does not walk through the implementation of every method it ships; each has its own optimization, inference theory, and proofs, which belong to the source papers and the per-estimator documentation (for example https://mlsynth.readthedocs.io/en/latest/sdid.html). Nor is anything special about the three estimators I illustrate below: none is privileged over the dozens I omit, and a different three would have served as well. They can be shown side by side, and swapped with a one-line edit, because they share one input contract and one interface.

1.1. Related software

Individually, the estimators in `mlsynth` are already well served by existing software. The canonical implementations live in R and Stata, split one method to a package; the few Python options each cover a narrow slice. I group the representative packages by strand, note the `mlsynth` equivalent, and report where I have checked the two numerically; Table 1 collects them.

The original synthetic control method ships as the R package `Synth` (Abadie *et al.* 2011), itself a paper in this journal, and is reproduced as **VanillaSC**. The numerics of its nested predictor-and-donor optimization have since been studied: the R package `MSCMT` (Becker and Klößner 2018) and the bilevel analysis of Malo *et al.* (2024) show the problem has a global optimum where the donor weights coincide with a pure outcome fit, which **VanillaSC**'s outcome-only backend reaches directly. On the California tobacco data it agrees with the installed `MSCMT` on every donor weight (Utah 0.3939, Montana 0.2318, Nevada 0.2049, Connecticut 0.1091) and on an average effect of −19.51363, to five decimals; run against the original `Synth` solver under outcome-only matching it matches the donor weights to about 0.003 while reaching a strictly lower value of the loss `Synth` minimizes, the summed squared pre-period residuals (52.130 against `Synth`'s 52.136; an RMSE of about 1.66), the small optimality gap the numerics literature describes, shown here against the reference implementation itself.

`synth2` (Yan and Chen 2023) re-implements the estimator in Stata with the inference practitioners want around it, placebo tests, a leave-one-out check, and confidence intervals. In `mlsynth` these are options on one estimator: **VanillaSC** with `inference = "placebo"` runs the in-space placebo and `inference = "scpi"` attaches the prediction intervals of Cattaneo, Feng, and Titiunik (2021). Two further options have no packaged Python implementation: `inference = "lto"` runs the leave-two-out refined placebo test of Lei and Sudijono (2025), far more powerful in small donor pools, and `inference = "ttest"` applies the debiased synthetic-control t-test of Chernozhukov, Wuthrich, and Zhu (2025), ported from the authors' R `scinference`.

A second strand replaces the simplex with a regression of the treated unit on the donors, the panel-data approach of Hsiao *et al.* (2012). The Stata package `rcm` (Yan and Chen 2022) and

the R package `pampe` (Vega-Bayo 2015) implement it with a menu of model-selection routines, and the R package `fsPDA` (Shi and Huang 2023) adds the forward-selected refinement. `mlsynth` carries the whole strand in **PDA**, whose `method` argument selects the Hsiao-Ching-Wan best-subset fit (`"hcw"`), forward selection (`"fs"`), the lasso, or an ℓ_2 relaxation. It reproduces `pampe`'s Hong Kong application (the original Hsiao *et al.* (2012) study), selecting the same donor set and effect path, and on `fsPDA`'s anti-corruption / luxury-watch application agrees with `fsPDA` to five decimals, selecting the same three donors and returning a monthly average effect of −0.030896 (−3.09%), the figure Shi and Huang (2023) report.

A third strand refines the simplex fit itself. `augsynth` (Ben-Michael *et al.* 2021) adds a ridge-penalized outcome model to correct imperfect pre-treatment fit; its augmentation is the `augment = "ridge"` option on **VanillaSC**, and on `augsynth`'s own Kansas tax-cut study the two agree to the sixth decimal (−0.029435 un-augmented, −0.040063 ridge-augmented). A fourth changes how donor weights are formed: `masc` (Kellogg, Mogstad, Pouliot, and Torgovitsky 2021) interpolates between nearest-neighbour matching and the SC simplex by a cross-validated mixing parameter, and `microsynth` (Robbins and Davenport 2021), a paper in this journal, calibrates weights to match baseline characteristics exactly for micro-level panels. `mlsynth` carries both: **MASC** reproduces the Basque study of Kellogg *et al.* (2021) (the cross-validation selects pure SC, the same Cataluna-plus-Madrid donors, an effect of −$585 against the paper's −$580), and **MicroSynth** reproduces the R package's calibrated solution on the Seattle drug-market study to the paper's table tolerance.

A final strand is Bayesian. `CausalImpact` (Brodersen, Gallusser, Koehler, Remy, and Scott 2015) and `mbsts` (Qiu, Jammalamadaka, and Ning 2018) fit state-space counterfactuals with the donors as regressors; `mlsynth` brings this route inside its interface as **CMBSTS**, a port of `CausalMBSTS` (Menchetti and Bojinov 2022) cross-checked cell by cell against the R reference. A second offering, **BVSS** (Xu and Zhou 2025), stays in the weighted-donor family, placing a spike-and-slab prior over the donors and a soft simplex constraint on their weights; its posterior reproduces the authors' own Gibbs sampler on their trade panel, the two agreeing on the posterior-mean effect to Monte-Carlo error. Both Bayesian ports are pinned as durable benchmark cases.

The Python options are fewer and narrower. `SyntheticControlMethods` (Engelbrektson 2020) implements the classic estimator; `pysyncon` (Fordham 2022) adds its robust, augmented, and penalized variants but no inference or experimental design; `CausalPy` (PyMC Labs 2024) places a Bayesian synthetic control alongside other quasi-experimental designs on a `PyMC` backend. Closest in spirit is `causaltensor` (Peng, Agarwal, and Shah 2022), which collects matrix-completion and factor-model estimators behind a common interface, the nearest peer to `mlsynth`'s missing-data family (**MCNNM**, **SNN**), but does not carry the simplex, regression-control, penalized, spillover-aware, or experimental-design estimators.

To my knowledge `mlsynth` is the first comprehensive Python library for this family: every estimator named above, and roughly forty more, is reached through one data-preparation step, one configuration-and-fit call, and one standardized result, where today a researcher who wants to weigh these methods against one another must cross R, Stata, and Python and reconcile as many data conventions and result formats. The numerical matches above are either re-run against the installed reference or reproduced from the source paper at validation time (Table 1).

Package	Language	Method implemented	In `mlsynth`	Checked
`Synth`	R	simplex SCM, predictor matching	**VanillaSC**	outcome-only: weights ~3e-3, lower SSR
`synth2`	Stata	simplex SCM, placebo and robustness tests	**VanillaSC**, `inference=`	n/a
`rcm`	Stata	regression control (subset, lasso, stepwise)	**PDA**, `method=`	n/a
`pampe`	R	Hsiao-Ching-Wan panel-data approach	**PDA**, `method="hcw"`	donor set, effect path
`fsPDA`	R	forward-selected PDA	**PDA**, `method="fs"`	−0.030896 (5 dp)
`augsynth`	R	augmented (ridge) SCM, covariates, staggered	**VanillaSC**, `augment="ridge"`	−0.0294/ −0.0401 (6 dp)
`MSCMT`	R	generalized / bilevel SCM	**VanillaSC**, outcome-only backend	installed package (5 dp)
`masc`	R	matching + synthetic control	**MASC**	−$585 vs −$580 (paper)
`microsynth`	R	calibrated SCM for micro-level panels	**MicroSynth**	weights vs R package
`causaltensor`	Python	matrix completion, factor models	**MCNNM**, **SNN**	n/a
`CausalImpact`	R	Bayesian structural time series	—	n/a
`mbsts`	R	multivariate Bayesian structural time series	**CMBSTS**	cell-by-cell vs `CausalMBSTS`
`CausalPy`	Python	Bayesian SC, RD, DiD, ITS (`PyMC`)	—	n/a
`tidysynth`	R	simplex SCM (tidyverse API)	**VanillaSC**	n/a
`SyntheticControlMethods`	Python	classic SCM	**VanillaSC**	n/a

Package	Language	Method implemented	In `mlsynth`	Checked
`pysyncon`	Python	classic / robust / augmented / penalized SCM	**VanillaSC**, **CLUSTERSC**	n/a
`mlsynth`	Python	all of the above and roughly forty more	one interface	n/a

Table 1: Representative synthetic-control packages and their `mlsynth` equivalents. The canonical implementations are spread across R and Stata and split one method to a package; the few Python options each cover a narrow slice. The final row is the contribution in one line: a single validated interface, in one language, over a family otherwise spread across three languages and a dozen packages. A dash in the fourth column marks an adjacent tool that targets a distinct counterfactual, which `mlsynth` does not reproduce. The "Checked" column records where `mlsynth`'s benchmark suite reproduces the reference implementation or the source paper's published result numerically; `n/a` there marks a package whose engine `mlsynth` reaches through an estimator cross-checked elsewhere rather than against that package directly (for instance the canonical `Synth` check covers the simplex backend that `tidysynth` and `SyntheticControlMethods` share), not an unverified claim. The full per-estimator validation is maintained on the validation dashboard (https://mlsynth.readthedocs.io/en/latest/validation.html).

The remainder of the paper is organized as follows. Section 2 develops the shared computational view and the resulting taxonomy of estimators. Section 3 describes the library's software design along the path one call takes: the Pydantic-validated configuration object (Colvin and Pydantic Contributors 2024), the one-package-per-estimator structure, the internal data-preparation routine, and the standardized result hierarchy. Section 4 then illustrates the library by problem regime: a cross-method comparison on the Proposition 99 tobacco panel, where convex, robust-matrix, and predictor-selecting synthetic controls are fit and read off together in one call, each with a matched prediction interval; a second comparison on the West German reunification, where a proximal, a robust-PCA, and a spillover-inclusive estimator – each previously confined to R or to research code – defend against a different threat to the fit and still agree on the effect; and experimental design, using MAREX to reproduce Abadie and Zhao's Walmart placebo design on real store data. Section 5 describes the library's testing, validation, and documentation, Section 6 concludes, and Section 7 records the computational and reproducibility details.

2. A unified view of synthetic control

2.1. The shared model

Consider a balanced panel of N units observed over T periods. Let y_{it} denote the outcome of

unit i in period t. A single treated unit, indexed $i = 1$, receives an intervention at time $T_0 + 1$; the remaining $N-1$ units, the *donor pool*, are never treated. Collect the donor outcomes in the $T \times (N-1)$ matrix $\mathbf{Y}_0$ and the treated unit's outcomes in the T-vector $\mathbf{y}_1$. The first T_0 rows are the pre-intervention window; the remaining $T_1 = T - T_0$ rows are the post-intervention window.

It helps to state the causal target in potential-outcomes terms (Rubin 1974; Imbens and Rubin 2015). Each unit and period has two potential outcomes: $y_{it}(1)$, the outcome were unit i exposed to the intervention in period t, and $y_{it}(0)$, the outcome were it not. Let $D_{it} = 1$ when unit i is under the intervention in period t and $D_{it} = 0$ otherwise; in the single-treated-unit design $D_{it} = 1$ exactly when $i = 1$ and $t > T_0$. What I observe is governed by the switching equation,

$$y_{it} \;=\; D_{it}\, y_{it}(1) \;+\; (1 - D_{it})\, y_{it}(0), \tag{1}$$

so a unit reveals its treated potential outcome only while it is treated and its untreated one the rest of the time. Only one of the two is ever seen in any cell, the fundamental problem of causal inference (Holland 1986). For the treated unit after T_0 I therefore observe $y_{1t} = y_{1t}(1)$, while the quantity the analysis needs, the untreated potential outcome $y_{1t}(0)$, is missing. The per-period effect on the treated unit is the gap

$$\tau_t \;=\; y_{1t}(1) - y_{1t}(0), \qquad t > T_0, \tag{2}$$

and the summary estimand is its average over the post-intervention window, the average effect on the treated ATT $= T_1^{-1} \sum_{t>T_0} \tau_t$ (read as an expectation when the effects are treated as random). Because $y_{1t}(1)$ is observed, the whole task reduces to imputing the missing $y_{1t}(0)$. The donor pool is never treated, so by Equation 1 its outcomes trace the untreated regime throughout; every estimator below borrows that information to fill in $y_{1t}(0)$, which the synthetic control $\hat{y}_{1t}$ estimates.

What the family shares at the level of theory is narrower than any single model, and is worth stating on its own: there exist donor weights $\mathbf{w}$ under which a combination of the donors reproduces the treated unit's untreated potential outcome, so that

$$y_{1t}(0) \;=\; \mathbf{Y}_{0,t}\, \mathbf{w} \;+\; r_t, \qquad t > T_0, \tag{3}$$

holds in the post-intervention window with an approximation error r_t that each method's theory drives to zero, even though the weights were fixed using only pre-intervention data. Granting such weights, the synthetic control

$$\hat{y}_{1t} \;=\; \mathbf{Y}_{0,t}\, \hat{\mathbf{w}}, \qquad t > T_0, \tag{4}$$

estimates the missing counterfactual, and the period-t effect is the gap $\hat{\tau}_t = y_{1t} - \hat{y}_{1t}$. The existence of $\mathbf{w}$ is the load-bearing assumption; what the methods disagree about is *why* such weights should exist and *how* to recover them. The library takes no position among these rationales, and it is worth setting the main four side by side.

The classical justification is a low-dimensional factor model (Abadie *et al.* 2010). Suppose the untreated outcome obeys

$$y_{it}(0) \;=\; \delta_t \;+\; \boldsymbol{\lambda}_i^{\top} \mathbf{f}_t \;+\; \varepsilon_{it}, \tag{5}$$

where $\mathbf{f}_t$ is an r-vector of unobserved common factors (with r much smaller than N), $\boldsymbol{\lambda}_i$ is unit i's loadings, δ_t is a common time effect, and ε_{it} is idiosyncratic noise. Units then differ

only through the handful of latent loadings, so if the treated unit's loadings lie in the span of the donors', weights that reproduce its pre-treatment outcomes also reproduce its loadings, and hence its untreated path afterward. But the factor model is sufficient for Equation 3, not necessary, and three further rationales reach the same existence claim from different premises:

- Proximal causal inference (Shi *et al.* 2026) keeps the latent factors but drops the requirement that the treated unit's loadings sit in the donors' convex hull. It treats the donor outcomes as imperfect, noisy stand-ins for the unobserved confounders $\mathbf{f}_t$, and uses the donors that do not enter the synthetic control to help pin down the weights of those that do. The weights are justified by this proxy structure rather than by the geometry of a convex hull, which lets synthetic control tolerate poor pre-treatment fit and handle binary or count outcomes, without requiring the factor model to be linear.
- Forecast combination treats the weights as a prediction device rather than a structural object (Zhang, Wang, and Zhang 2022). As the pre-period lengthens the synthetic-control weights converge to the combination that minimizes mean-squared prediction risk, and that limit is optimal, lowest prediction error among all estimators built from a weighted average of donors, even when the pre-treatment fit is imperfect and no factor model holds exactly. Here weights exist because a best forecast combination always does; whether it recovers latent structure is beside the point.
- Balancing makes the weights the solution to a set of moment constraints (Hainmueller 2012). One requires the weighted donors to match the treated unit on chosen characteristics, covariate means or outcome lags, and, among all weights that achieve this balance, selects the least dispersed by an information-theoretic criterion (entropy, empirical likelihood, an ℓ_2 divergence). Liao, Shi, and Zheng (2026) make this precise for synthetic control: their relaxation approach replaces exact matching with an approximate-balance constraint that keeps the weighted donors within a tolerance of the treated unit's characteristics, the relaxation level η in Equation 6 below, and shows the resulting weights stay well behaved even when exact balance is infeasible. Existence then becomes a feasibility statement about the balance constraints, and ties synthetic control to the wider reweighting literature for observational studies.

However different these justifications, the estimators they motivate share one computational shape. Each takes the treated unit's pre-treatment outcome vector $\mathbf{y}_1$ and the donor matrix $\mathbf{Y}_0$, solves an optimization that chooses how to combine the donors, uses the result to impute the missing counterfactual, and reports the gap between the two. Vector, matrix, optimization, weights, plot: that recurring shape, not any one statistical model, is what makes a single interface the right abstraction, and it is the contract that `dataprep` (Section 3.3) and the fit-and-result interface were built to encode. The four rationales thus converge on one computation: for almost every estimator in the library the weights solve a single optimization program. Writing $\mathbf{Y}_0$ and $\mathbf{y}_1$ now for their pre-treatment (T_0-row) blocks, the donor weights solve

$$\hat{\mathbf{w}} \in \underset{\mathbf{w} \in \mathcal{C}}{\arg\min}\ \mathcal{L}(\mathbf{Y}_0, \mathbf{y}_1, \mathbf{w}) + \mathcal{P}(\mathbf{w}) \qquad \text{subject to} \qquad \mathcal{B}(\mathbf{Y}_0, \mathbf{y}_1, \mathbf{w}) \leq \eta. \tag{6}$$

Here $\mathcal{L}$ is a loss measuring pre-treatment fit, $\mathcal{P}$ is a penalty that shapes the weights, $\mathcal{C}$ is the admissible set the weights must lie in, and $\mathcal{B} \leq \eta$ is an optional set of balance conditions with relaxation level η; either the loss or the balance operator may be absent, but not both.

The balance operator $\mathcal{B}$ deserves a word, because it is where the moment-matching rationale

enters and it is easy to mistake for the loss. Abadie's original estimator does not fit the outcome path at all: it chooses weights so that the synthetic unit matches the treated unit on a vector of pre-treatment characteristics $\mathbf{x}_1$, covariate averages together with outcome lags, under a weighted metric,

$$\min_{\mathbf{w} \in \mathcal{C}_\Delta} \; (\mathbf{x}_1 - \mathbf{X}_0\mathbf{w})^\top \mathbf{V} \, (\mathbf{x}_1 - \mathbf{X}_0\mathbf{w}), \tag{7}$$

with $\mathbf{V}$ a positive semi-definite matrix weighting the characteristics by their predictive importance (Abadie *et al.* 2010). Requiring $\mathbf{X}_0\mathbf{w} \approx \mathbf{x}_1$ is a balance condition: the synthetic unit must resemble the treated unit on the things that matter before it is allowed to stand in for it afterward. Fitting the pre-treatment outcome path, the loss $\mathcal{L}$, is then the special case in which the characteristics are the outcome lags themselves; balancing estimators keep the constraint but swap the squared metric for an information-theoretic divergence and relax exact matching to $\mathcal{B} \leq \eta$. Seen this way, $\mathcal{L}$ and $\mathcal{B}$ are two operationalizations of one demand, make the synthetic unit indistinguishable from the treated unit before the intervention, which is why Equation 6 carries both and insists on at least one.

Classical synthetic control (Abadie *et al.* 2010) is the specialization of Equation 6 with a squared-error loss $\mathcal{L} = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2$, no penalty ($\mathcal{P} = 0$), no separate balance constraint, and the admissible set fixed to the unit simplex $\mathcal{C}_\Delta = \{\mathbf{w} \geq \mathbf{0} : \mathbf{1}^\top\mathbf{w} = 1\}$, which yields an interpretable, sparse convex combination of donors. The rest of the family is, to a first approximation, a catalog of other choices for the four objects in Equation 6.

What then separates the estimators is which rationale they commit to and, with it, how the weights $\mathbf{w}$ (or, more generally, the projection onto the donor span) are obtained. Beyond the choice of admissible set and penalty in Equation 6, members of the family differ in how they treat the donor matrix itself:

- Penalized weights add a term that trades the fit of the synthetic control as a whole against how closely each contributing donor resembles the treated unit, singling out a unique, sparser solution when many donors would otherwise match equally well (Abadie and L'Hour 2021). Augmented weights instead fit an outcome model, typically a ridge regression, to the imbalance the simplex leaves in the pre-period and subtract the implied bias, which relaxes the convex-hull restriction and can place negative weights on some donors (Ben-Michael *et al.* 2021).
- Principal-component and robust variants first denoise $\mathbf{Y}_0$ by truncating its singular-value spectrum, estimating $\mathbf{f}_t$ directly; the thresholding doubles as donor selection and tolerates missing entries, and the weights are fit on the denoised matrix (Amjad *et al.* 2018), the low-rank reading of Equation 5.
- Forward-selection and best-subset approaches choose a small set of donors rather than weighting all of them: building on the panel-data method of Hsiao *et al.* (2012), a forward-selection algorithm adds control units one at a time and supports valid post-selection inference even when the candidate pool grows faster than the pre-period (Shi and Huang 2023).
- Factor and interactive-fixed-effects estimators fit Equation 5 explicitly (Bai 2009; Xu 2017), while matrix-completion methods recover the untreated-outcome matrix by low-rank completion under a nuclear-norm penalty, imputing the treated entries directly (Athey *et al.* 2021).

Seen this way, the catalog is a set of choices along a few axes (the constraint on the weights, whether the donor matrix is denoised first, whether donors are selected or weighted, how the common time structure is handled, and which justification for the weights from Equation 3 one is willing to defend), not a list of unrelated algorithms. Every one still consumes the same inputs ($\mathbf{Y}_0$, $\mathbf{y}_1$, T_0) and returns the same output, a counterfactual path and its gap, which is why the whole family fits behind one interface. Reduced to the same inputs and outputs, a factor-model estimator, a proximal one, a forecast-combination one, and a balancing one can be run on one panel and judged by the counterfactuals they produce rather than the premises they require.

Every estimator the library ships targets one of a few problem regimes – a single treated unit, low-rank and factor models, time series, distributional and multi-outcome settings, endogeneity and proxies, spillovers, staggered adoption, and experimental design – and each is guarded by a durable benchmark: the source paper's own empirical result (Path A), its Monte Carlo or simulation-table target (Path B), or a numerical cross-check against a reference implementation (X). Six classes are dispatchers that host several named methods behind one interface, so the number of distinct methods a user can run exceeds the number of classes. Rather than freeze that catalogue as a table that would drift out of date, the full per-estimator list and its live validation status are maintained on the validation dashboard (https://mlsynth.readthedocs.io/en/latest/validation.html), regenerated from the benchmark suite of Section 5; Section 4 illustrates three of the estimators in detail, and the dashboard is the map of everything else a one-line change can reach.

3. Software design

The library is organized so that each piece teaches the next. A user writes one configuration, hands it to one estimator, and reads one result; in between, the data are prepared, internally, identically, by one routine the user never calls. I follow that path (config, estimator, data preparation, result) because it is also the order in which the design builds on itself. Two commitments run through all four. Outwardly, every estimator presents the same surface, so learning one teaches every other. Inwardly, every estimator is its own self-contained package, which keeps more than forty of them maintainable.

3.1. One config

Each estimator is driven by a dedicated configuration object built on Pydantic (Colvin and Pydantic Contributors 2024), a runtime type-validation library. Configurations forbid unknown fields, document every option, and fail early with a typed error when an input is malformed, a missing treatment column, a treatment date outside the sample, an empty donor pool. There are no free-form keyword arguments. The usage pattern is uniform: construct a configuration (here as a plain dictionary, which Pydantic coerces and validates), pass it to the estimator class, and call `fit()`.

```
from mlsynth import VanillaSC

config = {
```

```
    "df": panel,            # a long/tidy pandas DataFrame
    "outcome": "cigsale",   # the outcome column
    "treat": "treat",       # 0/1 treatment indicator
    "unitid": "state",      # the unit column
    "time": "year",         # the time column
}
results = VanillaSC(config).fit()
```

The five required fields name the parts of the panel. `df` is the tidy long data frame itself, one row per unit and period. `outcome` names the column to be explained, the variable whose treated-minus-counterfactual gap is the object of interest. `treat` names a 0/1 indicator equal to one for the treated unit in its post-intervention periods and zero everywhere else; from it alone the package reads which unit is treated and when, so the intervention date and the pre/post split need never be stated separately. `unitid` and `time` name the columns that identify the unit and the period. Everything past these five is method-specific and optional, each carrying its own default.

Switching estimators means changing the class name and, at most, a handful of method-specific options; the data, the column names, and the call shape do not move. This is the concrete payoff of the unified view in Section 2.

Under the surface, this uniformity is enforced by type rather than convention. Every configuration descends from one of two base models. Observational estimators extend `BaseEstimatorConfig`, which declares the five required fields above; the experimental-design family extends `BaseMAREXConfig`, identical but for one telling omission, it requires no `treat` column, because a design chooses whom to treat rather than reading it from the data. That split in the type system is the two-family results contract of Section 3.4 seen from the input side. Both bases forbid any field they do not recognize and validate the panel on construction, and a specialized estimator extends its base with its own typed, documented options.

```
from mlsynth.config_models import BaseMAREXConfig
from mlsynth.utils.marex_helpers.config import MAREXConfig

design_fields = set(MAREXConfig.model_fields) - set(BaseMAREXConfig.model_fields)
n_design_fields = len(design_fields)
```

The design estimator MAREX is the fullest example. Its `MAREXConfig` inherits the design base and layers 47 further fields on top (a cluster column, per-stratum quotas, a budget, an adjacency matrix, lists of units to force into or out of treatment), each a declared field with a description and a validator, none of them a free-form keyword.

Because the fields are typed and the bases forbid the unknown, malformed input is rejected at construction, before any computation, and with a translated `mlsynth` exception rather than a raw trace. Naming an outcome column the data does not have is caught immediately:

```
import pandas as pd
from mlsynth import VanillaSC
from mlsynth.exceptions import MlsynthDataError

panel = pd.DataFrame({"state": ["A", "A", "B", "B"], "year": [1, 2, 1, 2],
```

```
                    "cigsale": [10.0, 9.0, 8.0, 7.0], "treat": [0, 1, 0, 0]})
try:
    VanillaSC({"df": panel, "outcome": "sales",        # no 'sales' column
               "treat": "treat", "unitid": "state", "time": "year"})
except MlsynthDataError as err:
    print(f"{type(err).__name__}: {err}")
```

```
MlsynthDataError: Missing required columns in DataFrame 'df': sales
```

Every estimator inherits the same discipline, so across the whole catalog a malformed analysis fails the same way: fast, typed, and legible.

Because a configuration is plain, validated data rather than live program state, the specification of an analysis is itself serializable. Everything in it but the DataFrame (the column names and every method-specific option) can be written to a JSON or YAML file, version-controlled, and loaded back to drive a fit. The record of *what* analysis was run thus lives separately from the data it ran on, which keeps to its own tabular format (CSV or Parquet); each can be inspected and diffed on its own. A reproducible study can therefore ship as a small text specification beside its data, with no bespoke script standing between the two. Concretely, the configuration above is a few lines of YAML,

```
# study.yaml: a complete, shareable analysis specification
estimator: VanillaSC
outcome:   cigsale
treat:     treat        # 0/1 treatment indicator
unitid:    state
time:      year
```

which `mlsynth` reads back into a ready-to-fit estimator, resolving the named method and attaching the data at load time:

```
from mlsynth import load_spec

est = load_spec("study.yaml", df=panel)   # resolves VanillaSC, attaches the panel
results = est.fit()
```

The library exposes the writing side symmetrically, as `save_spec`, so a fitted configuration can be exported to a record and shared.

That this takes only a few lines, and works for every estimator alike, is a dividend of the shared contract, not per-method plumbing: because a configuration is data, the estimator is named in its own record, and one `fit()` runs whatever loads, the uniformity that makes the catalog comparable also makes its analyses portable.

3.2. One estimator

That uniform surface is only affordable because the source is partitioned the same way the method catalog is. Each estimator is one package. A thin class in `estimators/<name>.py` does nothing but validate its configuration, hand off to a pipeline, translate any failure into the library's typed error vocabulary, and return the standardized result; **VanillaSC** is seventy-

odd lines and holds no algorithm of its own. The work lives beside it in a helper package, `utils/<name>_helpers/`, split by role, typically a setup step that prepares the data, the numerical pipeline, the data structures, a plotter, and whatever modules are specific to the method. The configuration object lives in that same package, in its own `config.py`, next to the code it governs, and is re-exported centrally so existing imports keep resolving.

The motivation is mundane and decisive. The alternative, the project's own earlier design, is a handful of shared modules that every estimator edits in common; a single configuration file had grown toward several thousand lines, becoming longer and riskier to touch with each method added. Giving every method its own package is the remedy. A contributor adding or revising an estimator works inside one directory: the change cannot reach another estimator's code, two people editing different estimators never collide, and the blast radius of a mistake is one package. The same boundary serves the reader. The unit one must hold in mind to follow a method, its options, its validation rules, its numerics, and its tests, is exactly one directory, not a thread chased across a monolithic configuration module, a monolithic estimator module, and a shared helper file.

What stays shared is deliberately small and cross-cutting, and most of it was met already in Section 3.1: every configuration inherits one base that fixes the common `df / outcome / treat / unitid / time` surface, so a method's own `config.py` declares only what is genuinely specific to it. Two further commitments are shared the same way. Every failure is raised in one hierarchy, `MlsynthConfigError`, `MlsynthDataError`, and `MlsynthEstimationError` beneath a common `MlsynthError`, so that user code catches the same types no matter which estimator raised them. And when a method is really a family of related variants, the package gains a dispatcher rather than the library gaining several near-duplicate estimators: **SPILLSYNTH** selects among five spillover-adjustment schemes through a single `method` argument, each isolated in its own subpackage. The rule scales the catalog without scaling the coupling.

3.3. One dataprep

Inside that thin estimator class, the first step is data preparation, the one part of the library the analyst never touches. `mlsynth` asks only for a clean, tidy panel, a long data frame, one row per unit and period, prepared before any estimator sees it. Ingestion is then routed through a single internal routine, `dataprep()`, which the user never calls. It takes the tidy frame together with the names of its outcome, unit, time, and treatment columns and returns a canonical bundle, the wide outcome matrix, the treated vector $\mathbf{y}_1$, the donor matrix $\mathbf{Y}_0$, and the pre- and post-treatment period counts T_0 and T_1. Every estimator consumes that same bundle, so every data frame is pivoted, aligned, and validated the same way, every single time, in one place that is written and tested once. What varies is only what the analysis requires (whether covariates are supplied, say), never the mechanics of getting a data frame into an estimator.

The implication worth dwelling on is what the user does *not* have to specify. A single binary treatment indicator carries all the structure `dataprep()` needs: the unit whose indicator ever turns on is the treated unit, the period in which it first turns on is $T_0 + 1$, and everything before that is the pre-treatment window. The user never names the treated unit, never states the intervention date, and never declares the pre/post split; these are read off the indicator. The same convention scales without new arguments: when several units are treated at different

times, `dataprep()` cohorts them by their individual adoption dates automatically.

This is a deliberate contrast with the surrounding ecosystem, and it shows first as sheer simplicity at the call site. To get from a data frame to an estimated, plotted result with the widely used `scpi` package (Cattaneo *et al.* 2025), the analyst imports six separate functions and chains four of them by hand, `scdata` to prepare and reshape the data, then `scest` to estimate, `scpi` to do inference, and `scplot` to draw the result, with the data-preparation step, `scdata`, squarely the analyst's job. `tidysynth` (Dunford 2021), the most ergonomic of the R options, still asks for a pipeline of four verbs, `synthetic_control`, `generate_predictor`, `generate_weights`, and `generate_control`, and then separate `grab_*` and `plot_*` calls to recover the weights, the counterfactual, and the figures. The forward-selection `fsPDA` implementation (Shi and Huang 2023) requires the data to be pre-pivoted into wide treated-versus-donor matrices before it will run at all. In `mlsynth` the analyst imports one estimator, builds one configuration, and calls `fit()` once; there is no preparation function to find, import, configure, or call. Coding the indicator column *is* the specification, and the same long data frame drives every one of the more than forty estimators unchanged.

3.4. One result

Every `fit()` returns a typed result object drawn from a single hierarchy. Observational estimators return an effect result; design methods return a design result whose report is itself an effect result. Both populate the same standardized sub-objects, estimated effects, fit diagnostics, the observed and counterfactual time series, donor weights, and inference, as typed fields rather than ad hoc dictionaries. Reading a result is therefore the same regardless of which estimator produced it:

```
res = VanillaSC(config).fit()

res.effects.att                           # average effect on the treated
res.time_series.counterfactual_outcome    # the synthetic control path
res.weights.donor_weights                 # the fitted donor weights
res.inference.p_value                     # inference, on the same object
```

Table 2 names the six standardized sub-objects and what each holds; an estimator populates whichever apply to it and leaves the rest empty, so the same field access works across the catalog.

Sub-object	What it carries
`effects`	the ATT, its percentage version, and its standard error
`fit_diagnostics`	pre- and post-treatment RMSE and the pre-period R^2
`time_series`	the observed and counterfactual paths, their gap, and the time axis
`weights`	donor weights, and where they apply, time weights or a unit-weight matrix

Sub-object	What it carries
`inference`	the p-value, confidence interval, standard error, and inference method
`method_details`	the variant that produced the result and the parameters it used

Table 2: The standardized result sub-objects shared by every estimator. Beyond these, two typed escape hatches carry anything a method needs that the schema does not name.

The same fields, read off a real fit, return the numbers themselves. Here is **VanillaSC** on the Basque Country data (Abadie and Gardeazabal 2003), using the outcome-only backend and the debiased synthetic-control t-test of Chernozhukov *et al.* (2025):

```
pd.Series({
    "effects.att":         round(res.effects.att, 4),          # average effect
    "effects.att_percent": round(res.effects.att_percent, 2),  # as a percentage
    "inference.p_value":   round(res.inference.p_value, 4),    # debiased t-test
    "inference.ci_lower":  round(res.inference.ci_lower, 4),
    "inference.ci_upper":  round(res.inference.ci_upper, 4),
    "inference.method":    res.inference.method,
})
```

```
effects.att                                                  -0.6915
effects.att_percent                                             -8.1
inference.p_value                                              0.042
inference.ci_lower                                           -1.2561
inference.ci_upper                                           -0.0589
inference.method       debiased SC t-test (Chernozhukov-Wuthrich-Zhu ...
dtype: object
```

Because the result shape is fixed, downstream code (plotting, tables, comparison across estimators) is written once and works for every method. The illustrations below read effects, time series, and inference off this common object without any estimator-specific glue.

4. Applications

The next three subsections apply the library to two cross-method comparisons and an experimental design. Each comparison fits three estimators on one panel through a single `compare_estimators` call – the first on the Proposition 99 tobacco panel, the second on the West German reunification – and the design example uses MAREX. Each is a self-contained, reproducible transcript.

4.1. Three traditions on one panel

I open with the study that made the method famous, and use it to make the paper's central claim concrete. Abadie *et al.* (2010) estimated the effect of California's 1988 Proposition 99 tobacco-control program by building a synthetic California from the other states, and put the decline at roughly nineteen packs per capita a year. The fifteen years since have produced estimators that answer that same question from very different premises, and I fit three of them, one from each of three traditions, on the one panel. The first is **VanillaSC**, the convex synthetic control of Abadie *et al.* (2010), whose non-negative donor weights sum to one and never reach beyond the donors' hull; I fit it on Abadie's own predictor set through the corner solver of Malo *et al.* (2024). The second is **CLUSTERSC** in its robust form (Amjad *et al.* 2018). The third is **SDID**, synthetic difference-in-differences (Arkhangelsky *et al.* 2021), which pairs donor weights with a difference-in-differences adjustment, matching the treated unit's pre-period trend rather than its level.

They share nothing in their internals and everything in their interface, so one call fits all three and lines their counterfactuals up against the treated series:

```
prop99 = pd.read_csv(find_data("augmented_cali_long.csv"))
prop99 = prop99[prop99["year"] <= 2000].copy()
prop99["treated"] = ((prop99["state"] == "California")
                     & (prop99["year"] >= 1989)).astype(int)
# Abadie's special predictors: three cigarette-sales lags and four covariates.
for L in (1975, 1980, 1988):
    prop99[f"cig{L}"] = prop99["state"].map(
        prop99.loc[prop99["year"] == L].set_index("state")["cigsale"])
covs = ["loginc", "p_cig", "pct15-24", "pc_beer", "cig1975", "cig1980", "cig1988"]
windows = {"loginc": (1980, 1988), "p_cig": (1980, 1988), "pct15-24": (1980, 1988),
           "pc_beer": (1984, 1988), "cig1975": (1975, 1975),
           "cig1980": (1980, 1980), "cig1988": (1988, 1988)}
p99 = dict(df=prop99, outcome="cigsale", treat="treated", unitid="state",
           time="year", display_graphs=False)

# Each estimator is configured however it likes; the comparison only asks that
# they share the panel. show_bands is a required choice, here left off.
cmp99 = compare_estimators(
    {"VanillaSC": VanillaSC({**p99, "backend": "malo",
                             "covariates": covs, "covariate_windows": windows}),
     "Robust SC": CLUSTERSC({**p99, "method": "PCR", "clustering": False}),
     "SDID":      SDID({**p99, "intercept_adjust": True})},
    show_bands=False)
s99 = cmp99.summary

fig, ax = plt.subplots(figsize=(5, 3.2))
cmp99.plot(ax=ax,                          # one call: three counterfactuals, one axis
           colors={"VanillaSC": "#4e6e8e", "Robust SC": "#a0785a",
                   "SDID": "#70896f"})
ax.axvline(1989, color="0.4", lw=1.0)
ax.set_xlabel("year"); ax.set_ylabel("cigarette sales (packs per capita)")
fig.tight_layout(); plt.show()
```

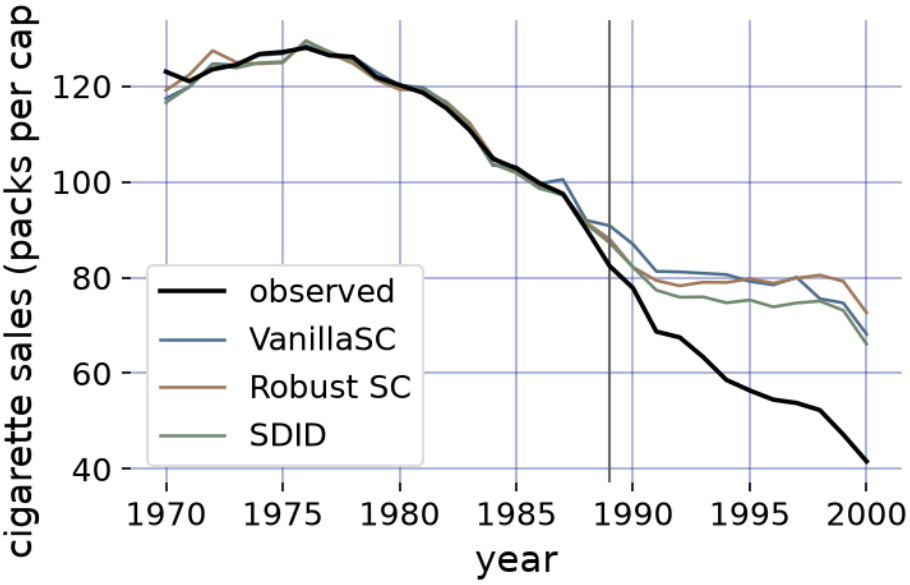

Figure 1: Three estimators of the Proposition 99 effect, fit on one panel: observed California cigarette sales (black) against VanillaSC (blue), Robust SC (brown), and SDID (green). The line marks the 1989 intervention.

The three estimators build California from different premises (Figure 1). VanillaSC, matching on Abadie's predictors through the Malo corner solver, puts the average post-1989 shortfall at 19.48 packs per capita, reproducing the value the R package `Synth` and the `MSCMT` solver return on this panel to the decimals reported in Section 1.1; Robust SC, fitting a low-rank donor matrix, returns almost the same, 19.37. Synthetic difference-in-differences returns a smaller 15.61, the estimate Arkhangelsky *et al.* (2021) report for this panel; it is smaller by construction, because its difference-in-differences step relaxes the level-matching the two synthetic controls enforce and asks the donors to match only the treated unit's pre-period trend. All three track California closely before 1989 (pre-period RMSE 1.66, 1.69, and 1.80), then separate after it. Reading three lineages – convex weighting, low-rank projection, and a difference-in-differences hybrid – off one panel in a single call, before one interface carried them, meant three packages, three data formats, and three notions of a result reconciled by hand. Only the interface is shared; what varies is the estimator, so the spread between the three is attributable to their assumptions alone, each cross-validated against its own reference implementation (Section 1.1).

4.2. Three defenses on one panel

Synthetic control rests on assumptions that observational panels often strain. The donor outcomes may be noisy proxies for the forces that move the treated unit; the donor pool may include units that do not belong or are contaminated by outliers; and a donor may itself be affected by the treatment, violating the no-interference assumption. `mlsynth` carries a dedicated estimator for each, and I apply all three to one panel.

Abadie, Diamond, and Hainmueller (2015) asked what the 1990 reunification cost West German

GDP per capita, building a synthetic West Germany from other OECD economies and tracing a shortfall that widened across the 1990s. I answer the same question three ways, each estimator addressing one of the three problems above. The first is **PROXIMAL** in its over-identified form (`PIOID`) (Shi *et al.* 2026), which reads the donor outcomes as error-laden proxies of West Germany's untreated path and uses a second, disjoint set of donor countries as instruments to purge the measurement error, a proximal-causal-inference repair for noisy proxies. The second is **CLUSTERSC** in its robust-PCA form (`rpca`) (Bayani 2021), which first selects the donor pool by clustering the units on their pre-period principal-component scores, keeping only those that move with the treated unit, then splits that pool into a low-rank signal and a sparse-outlier part by principal component pursuit and fits on the denoised signal, a repair for a donor pool that is ill-chosen or contaminated by outliers. The third is **SPILLSYNTH**'s inclusive synthetic control (`iscm`) (Di Stefano and Mellace 2024), which keeps Austria, the neighbour most exposed to reunification, in the pool and solves it jointly with West Germany, de-contaminating the spillover rather than discarding the donor, a repair for a no-interference violation.

Each was, until now, reachable only as its authors' own research code: the over-identified proximal estimator in the manuscript repository for Shi *et al.* (2026), the robust-PCA control in Bayani's dissertation scripts, and the inclusive method in Di Stefano and Mellace's replication files – three languages of result, none installable off the shelf. `mlsynth` carries all three behind one interface, so a single call fits them and reads their counterfactuals off together:

```
germany = pd.read_csv(find_data("german_reunification.csv"))
germany["treated"] = ((germany["country"] == "West Germany")
                      & (germany["year"] >= 1990)).astype(int)
wg = dict(df=germany, outcome="gdp", treat="treated", unitid="country",
          time="year", display_graphs=False)

# PIOID alone needs a proxy/instrument split of the donor pool: the six countries
# that carry synthetic West Germany act as proxies, the rest as instruments.
proxies = ["Austria", "Italy", "Japan", "Netherlands", "Switzerland", "USA"]
instruments = [c for c in germany["country"].unique()
               if c not in proxies + ["West Germany"]]

cmpwg = compare_estimators(
    {"PIOID": PROXIMAL({**wg, "donors": proxies, "outcome_instruments": instruments,
                        "methods": ["PIOID"]}),
     "RPCA-SC": CLUSTERSC({**wg, "method": "rpca", "rpca_method": "PCP"}),
     "Inclusive SCM": SPILLSYNTH({**wg, "method": "iscm",
                                  "affected_units": ["Austria"]})},
    show_bands=False)
swg = cmpwg.summary

fig, ax = plt.subplots(figsize=(5.2, 3.2))
cmpwg.plot(ax=ax,                          # one call: three counterfactuals, one axis
           colors={"PIOID": "#4e6e8e", "RPCA-SC": "#a0785a",
                   "Inclusive SCM": "#70896f"})
ax.axvline(1990, color="0.4", lw=1.0)
```

```
ax.set_xlabel("year"); ax.set_ylabel("GDP per capita (2002 US$)")
fig.tight_layout(); plt.show()
```

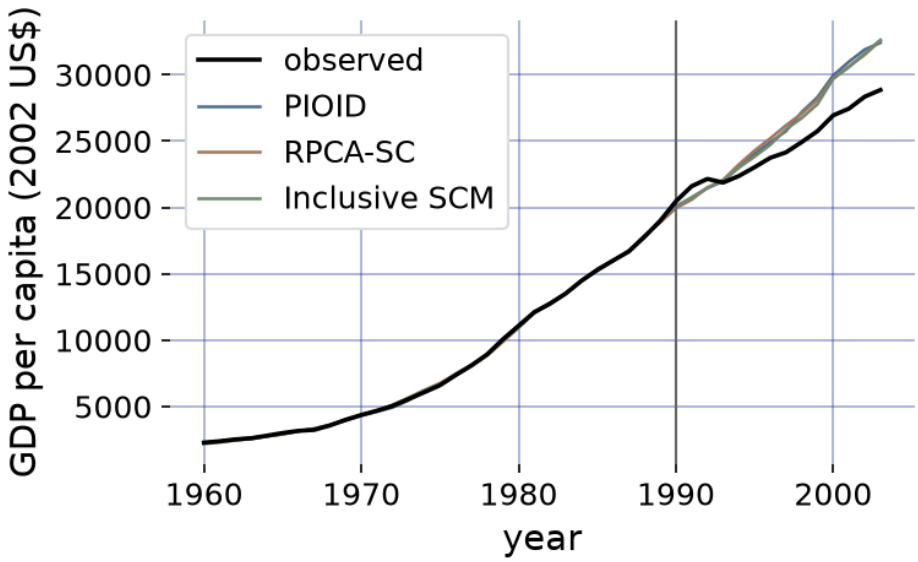

Figure 2: Three estimators of the 1990 West German reunification effect, fit on one panel: observed West German GDP per capita (black) against PIOID (blue), RPCA-SC (brown), and Inclusive SCM (green). The line marks reunification.

The three repairs disagree about what threatens the fit and agree about the answer (Figure 2). The over-identified proximal estimator, correcting the donor proxies with a separate block of instrument countries, puts the average post-1990 loss at \$1497 a head a year; the robust-PCA control, fitting West Germany to a donor matrix stripped of its sparse outliers, returns \$1501; and the inclusive control, which refuses to drop Austria and de-contaminates the spillover instead, returns \$1368. The inclusive figure is the more negative for a reason the method makes explicit: a synthetic West Germany that quietly borrows from an Austria depressed by the same shock understates the loss, and putting Austria back in on its own terms restores it. That three estimators guarding against three unrelated pathologies land within a couple of hundred dollars of one another is the robustness the reunification literature has long asserted and rarely shown side by side, because showing it meant three research codebases in two languages.

Only the interface is shared. Each fit is cross-validated against its originating implementation – the proximal estimator against the authors' manuscript code, the robust-PCA control against Bayani's own scripts, the inclusive control against Di Stefano and Mellace's – and those checks are pinned as durable benchmarks (`benchmarks/cases/proximal_germany_oid.py`, `clustersc_rpca_germany.py`, `spillsynth_iscm_germany.py`), so the agreement in Figure 2 is a fact about the methods rather than reconciled code.

4.3. Experimental design

The first two illustrations were *retrospective*. The treatment had already happened, and the

estimator's job was to reconstruct, after the fact, the counterfactual path of a unit that was treated whether or not a good comparison existed for it. The third regime inverts the order of operations. Here the experiment has not run yet, and the question is one of *design*: of all the units I could treat, which ones should I, and which should I hold back as controls, so a credible comparison exists once the experiment is over? The data are the same shape as before, a panel of units over time, but the assignment is now the thing being chosen rather than given, and choosing it well is what makes the eventual estimate trustworthy.

The hard part of a design is choosing which units to treat. When an experimenter can intervene in only a handful of large, heterogeneous units and must pick them and their controls before the experiment runs, splitting at random is unbiased on average, but a single random split can land the biggest units on one side and compromise the comparison before it begins. MAREX (Abadie and Zhao 2026) instead chooses the treated and control units up front, so that each group reproduces the population on its pre-period predictors; the paper proves this non-randomized design sharply reduces the bias of the effect later estimated, and to my knowledge `mlsynth` is the first package to make it available off the shelf.

I reproduce the authors' own empirical illustration (their Section 4): a *placebo* experiment on a balanced panel of weekly sales for forty-five Walmart stores over 143 weeks (Prakash 2023). The intervention is fictitious, so no store is actually treated, and a sound design must therefore produce synthetic treated and control units that track closely before the experiment and an estimated effect indistinguishable from zero. A placebo tests the design where it must not fail: it should not manufacture an effect that is not there.

Choosing the two groups is combinatorial: each unit may join the treated group, the control group, or neither, and the two synthetic units must each reproduce the population predictor mean. Write $\mathbf{x}_j$ for unit j's predictors (its pre-period outcomes together with any covariates), $\bar{\mathbf{x}} = \frac{1}{N}\sum_j \mathbf{x}_j$ for the population mean, $\mathbf{w}$ and $\mathbf{v}$ for the treated and control weights, and $z_j \in \{0,1\}$ for whether unit j is treated. The base (`standard`) design solves

$$\begin{aligned} \min_{\mathbf{w},\mathbf{v},\mathbf{z}} \quad & \left\|\bar{\mathbf{x}} - \textstyle\sum_j w_j \mathbf{x}_j\right\|_2^2 + \left\|\bar{\mathbf{x}} - \textstyle\sum_j v_j \mathbf{x}_j\right\|_2^2 \\ \text{s.t.} \quad & \textstyle\sum_j w_j = \sum_j v_j = 1, \quad w_j, v_j \geq 0, \\ & w_j \leq z_j, \quad v_j \leq 1 - z_j, \end{aligned} \tag{8}$$

with the number of treated units bounded by $m_{\min} \leq \sum_j z_j \leq m_{\max}$ (or fixed to an exact count m). The constraints $w_j \leq z_j$ and $v_j \leq 1 - z_j$ make a unit treated or control but never both, while the objective drives each synthetic unit's predictors onto the population's. This is a mixed-integer quadratic program (the binary $\mathbf{z}$), which `mlsynth` solves with the open-source SCIP solver (the paper used commercial Gurobi). Once the weights are fixed, the per-period effect is the gap between the two synthetic units, $\widehat{\tau}_t = \sum_j w_j y_{jt} - \sum_j v_j y_{jt}$. The `MAREX` class also exposes the paper's other objectives through one `design` argument (`weakly_targeted`, `penalized`, `unit_penalized`); I use the base design here.

I hand the authors' own Walmart panel to MAREX. It carries weekly sales for forty-five stores together with four store-level covariates an experimenter can also balance on, the local average temperature, fuel price, consumer price index, and unemployment rate (Prakash 2023). I design a placebo experiment with a fictitious intervention at week 129, taking the first hundred

weeks as the fitting window, the next twenty-eight as held-out blank weeks, and the final fifteen as the experimental period, and fix the number of treated stores to two, the count the paper adopts (one store fits poorly, three barely improves it):

```
from mlsynth import MAREX
from mlsynth.utils.marex_helpers.plotter import plot_marex

walmart = pd.read_csv(find_data("walmart_weekly_sales_covariates.csv"))  # 45 stores x 143
T = walmart["week"].nunique()
covs = ["Temperature", "Fuel_Price", "CPI", "Unemployment"]  # store-level covariates

experiment = MAREX({
    "df": walmart, "outcome": "sales", "unitid": "store", "time": "week",
    "T0": 128, "blank_periods": 28, "T_post": 15,   # 128 pre weeks: fit 1..100, blank 101
    "m_eq": 2,                                       # exactly two treated stores
    "covariates": covs, "standardize": True,         # match on sales and covariates
    "design": "standard", "inference": True, "display_graph": False,
}).fit()

treated = sorted(int(s) for s in experiment.treated_units)
w = experiment.globres.treated_weights_agg          # treated weights, one per store
v = experiment.globres.control_weights_agg          # control weights
```

The panel holds forty-five stores over 143 weeks. MAREX reads only the pre-experimental window to design, never the experimental sales, and matches each synthetic unit to the population on both the pre-period sales and the four covariates. It selects stores [15, 31] as the two treated units and builds a synthetic control for them from the remaining forty-three. A single `fit` does both jobs: the chosen stores' pre-period sales are untouched by the fictitious intervention, so reading the one panel through to the experimental weeks turns the design straight into the analysis.

A design is only as good as its balance. Figure 3 is a love plot of the four covariates: it contrasts the raw gap between the treated and control stores with each synthetic unit against the population, the quantity Equation 8 drives to zero.

Figure 4 is MAREX's own diagnostic of the experiment, drawn by the library. Through the fitting window and the held-out blank weeks the synthetic treated and synthetic control series track each other closely, and after the placebo intervention at week 129 the gap between them stays small, because there is no effect to find.

```
plot_marex(experiment, plot_type="prediction", donor_cloud=True, figsize=(7, 3.6))
```

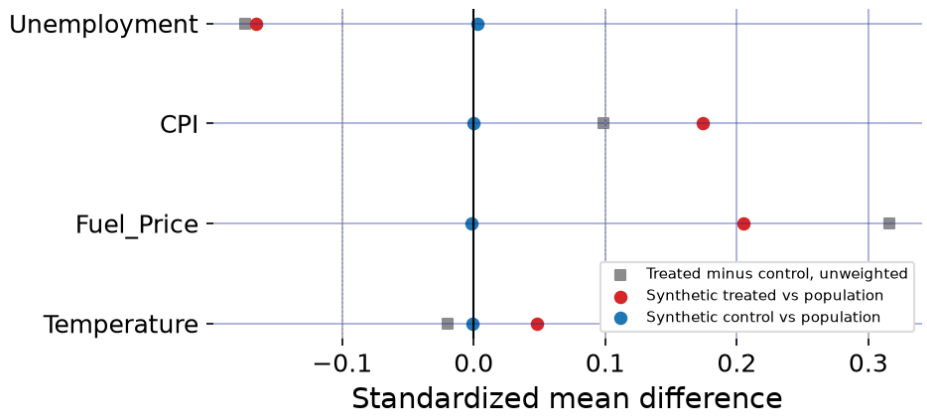

Figure 3: Covariate balance for the chosen design (a love plot), one row per store-level covariate. Grey squares are the raw standardized difference between the two treated stores and the controls before weighting; the red and blue dots are the synthetic treated and synthetic control, each against the population. The synthetic control (blue), free to draw on all forty-three remaining stores, sits on the population; the synthetic treated (red), restricted to two stores by the cardinality constraint, is pulled most of the way from the raw split toward balance.

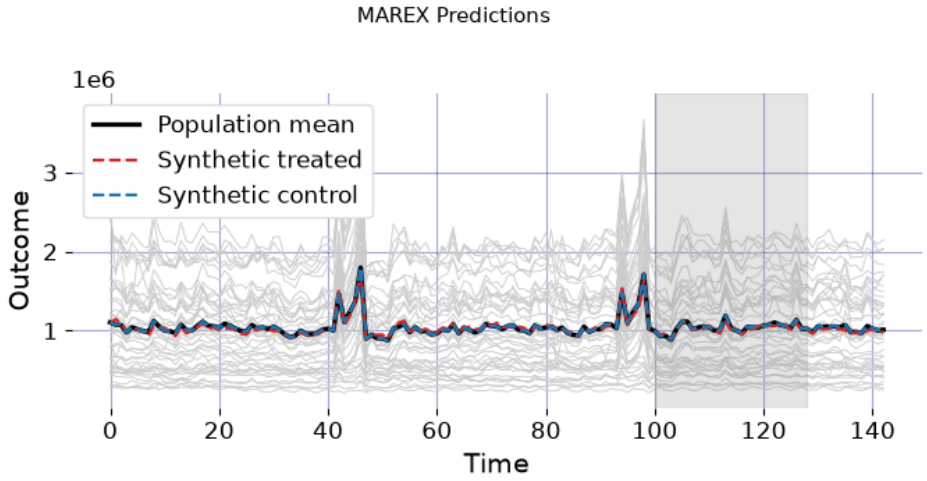

Figure 4: MAREX's own design diagnostic, drawn by the library, for the Walmart placebo experiment, reproducing Abadie and Zhao's Figure 2. The red and blue dashed lines are the synthetic treated and synthetic control units the design built from the pre-experimental window; the faint lines are the individual stores. The two synthetics track each other through the fitting and blank weeks and stay close after the fictitious intervention at week 129: the design produces no spurious effect.

Because the intervention is a placebo, the test of a good design is that it finds no effect, and it does. The synthetic treated and control units track to a pre-experimental fit RMSE of 2.9% of mean sales, and the estimated experimental effect is statistically indistinguishable from zero. The result reads like every other estimator's; because MAREX designs an experiment it

returns a *design* result, whose `report` is the realized effect and whose `post_fit` carries the diagnostics.

```
pd.Series({
    "report.att":           round(experiment.report.att, 1),   # placebo effect (sales uni
    "post_fit.ate_percent": round(pf.ate_percent, 2),          # ... as a percentage of me
    "post_fit.rmse_fit":    round(pf.rmse_fit, 1),             # pre-period tracking
    "post_fit.p_value":     round(pf.p_value, 3),              # permutation test
    "post_fit.ci_lower":    round(pf.ci_lower, 1),             # conformal band
    "post_fit.ci_upper":    round(pf.ci_upper, 1),
})
```

```
report.att             -28594.600
post_fit.ate_percent       -2.780
post_fit.rmse_fit       30371.200
post_fit.p_value            0.125
post_fit.ci_lower      -72513.400
post_fit.ci_upper       15324.300
dtype: float64
```

The placebo effect is -2.8% of mean sales with a confidence band that covers zero, and the permutation test fails to reject the null of no effect ($p = 0.12$): the design manufactures no effect where there is none. `mlsynth` reproduces the authors' own design code cell-by-cell. On this full 45-store panel, MAREX's mixed-integer solve and SCDesign's open cardinality-constrained routine (`quadprog`, no commercial Gurobi) select the same two stores, with treated weights agreeing to 2×10^{-4}, the same pre-period fit, and the same placebo effect to 10^{-4} of mean sales – pinned in the durable benchmark `marex_walmart`. Abadie and Zhao's headline placebo p-value of 0.933 is the no-covariate special case; matching on the four covariates here tightens the design and lowers the placebo p, as it does in the authors' own covariate runs.

Inference is the other half of the paper's contribution, and `mlsynth` carries it faithfully. Both the permutation p-value just quoted and the confidence band around each period's effect are built on the *blank* periods, pre-experimental weeks deliberately withheld from the fit, where the treated-minus-control gap is pure noise and so calibrates the size of a null fluctuation. Writing $\mathcal{B}$ for those blank weeks, the band is the experimental-period effect plus or minus the $(1-\alpha)$ empirical quantile of the held-out gaps,

$$\widehat{\tau}_t \;\pm\; q_{1-\alpha}, \qquad q_{1-\alpha} = \widehat{Q}_{1-\alpha}(\{\,|\widehat{\tau}_s| : s \in \mathcal{B}\,\}), \tag{9}$$

a split-conformal interval (Lei, G'Sell, Rinaldo, Tibshirani, and Wasserman 2018; Vovk, Gammerman, and Shafer 2005) calibrated on data the weights never saw. The construction is deliberately not the conformal method of Chernozhukov, Wüthrich, and Zhu (2021): where that procedure rearranges over all periods, including the pre-intervention window used to fit the weights, Abadie and Zhao permute only over the blank and experimental periods. The payoff is inference that stays valid under serial dependence and non-stationarity, rather than assuming the gaps are i.i.d. over time, which matters because store sales are persistent. Within that same framework `mlsynth` also surfaces a power analysis: an analytical minimum detectable effect, read off the blank-period residuals and inflated for their serial correlation by

an AR(1) variance-inflation factor, exposed on `res.post_fit.power` as a headline number and a curve over candidate horizons, so a designer can ask in advance how large an effect the experiment could detect and how long it must run (the derivation is on the estimator's documentation page, https://mlsynth.readthedocs.io/en/latest/marex.html). Design, estimate, permutation test, conformal band, and power curve all fall out of one `fit`, the first time, as far as I know, that this design-and-analyze workflow has been packaged for applied use.

5. Code quality and documentation

`mlsynth` is built to be stable, consistent, and inspectable. One data-preparation routine, one runtime-validated configuration object, and one standardized result stand in front of every estimator (Section 3), so a single set of coding and naming conventions holds across more than forty methods and the interface a user learns once transfers to all of them. Each estimator is a thin class over a dedicated helper package, which keeps the code for a method, its numerics, and its tests in one directory rather than threaded through a shared core. Configurations are validated with Pydantic (Colvin and Pydantic Contributors 2024): they refuse unknown or ill-typed fields at construction, so a mis-specified analysis fails as an immediate, legible error rather than a silent miscomputation. The package is free and open-source under the MIT license; it depends only on the scientific-Python stack, with the two heavier solver dependencies it can use loaded lazily so that the base installation runs every estimator. It is available from the Python Package Index via `pip install mlsynth`, with source and an issue tracker at https://github.com/jgreathouse9/mlsynth, and contributions and bug reports from the applied community are encouraged.

To keep the library correct as it grows, every estimator ships with automated tests, written before the code they cover and run in a continuous-integration environment on each change. Correctness against the literature is checked separately, by a benchmark suite of one hundred small re-runnable cases that hold each estimator to an outside answer rather than to its own past output. When an authoritative implementation of a method can be run, usually an R package or the original authors' replication code, the benchmark runs it on the same input and records its output, together with the input checksums and the exact package versions that produced it, as a captured reference the library is then pinned to; when no such implementation can be run, the benchmark instead targets a figure the source paper itself reports, its headline result or a cell of its simulation table. Because every target is either a re-runnable reference or a published fact, a regression in `mlsynth` or a drift in a reference surfaces as a failed benchmark rather than a quiet disagreement. Running the references, rather than transcribing their printed numbers, has a further benefit: each discrepancy is traced to its cause, and in several cases `mlsynth` is found to reach the verified optimum of the estimation problem where a published reference solver, at its default tolerances, does not.

The package is documented at https://mlsynth.readthedocs.io. Every estimator has its own page, organized the same way: when to use the method, the notation and model it assumes, its identifying assumptions each paired with a remark, a runnable example, and a pointer to the benchmark or replication that validates it. A decision tree guides the choice among methods for a given design, and a set of dedicated replication pages records, method by method, how the library's numbers were reconciled with their sources. The documentation is regenerated from the source for each release, and remains the up-to-date description of the

project's functionality as new estimators are added.

6. Summary

Synthetic control has grown from a single method into a sprawling family, and the software has fragmented along with it. I have argued the fragmentation is largely incidental: most of these estimators run one computation, differing only in how they constrain or denoise a single shared fit (Section 2). `mlsynth` takes that observation seriously, placing more than forty estimators behind one data contract, one validated configuration-and-fit interface, and one standardized result. The three illustrations show what the consolidation buys: a cross-method comparison of convex, robust-matrix, and predictor-selecting synthetic controls on Proposition 99, fit and read off together in one call; a second comparison on the West German reunification, where proximal, robust-PCA, and spillover-inclusive estimators – each previously available only in R or as research code – guard against different threats to the fit and still agree on the effect; and an experimental design with MAREX, the first packaged synthetic-control design, reproducing Abadie and Zhao's Walmart placebo on real data through the same interface as every estimator before it. Once a dozen estimators report in one format on one panel, the applied question can shift from which method is right to how far the answer depends on the method at all, a robustness check across estimators that `mlsynth` makes a one-line change.

The library has limits worth stating plainly. It consolidates existing methods rather than improving any one of them: for a single estimator a dedicated package may expose options `mlsynth` does not, and the library reproduces these methods rather than re-deriving or extending them. Coverage is uneven across regimes — most estimators target a single treated unit, with a smaller set for staggered adoption and for experimental design — and a few of the newest additions are still awaiting a durable benchmark case (flagged on the validation dashboard). The Bayesian and design-search estimators are markedly slower than the convex-program ones, and the weight solves scale with the size of the donor pool, so very large panels carry a cost. Inference is method-specific by design: each estimator carries the procedure its own theory supports rather than one uniform scheme. None of these is fundamental, and the shared interface makes each a contained target for later work; I record them so the catalog's breadth is not mistaken for uniform depth.

7. Computational details

The results above were produced with `mlsynth` 1.0.0 on Python 3.13.14, with NumPy 2.5.1 (Harris *et al.* 2020) and pandas 3.0.3, on Linux-6.17.0-1018-azure-x86_64-with-glibc2.39. Many convex weight problems are solved with `CVXPY` (Diamond and Boyd 2016). The package and its dependencies are open source; `mlsynth` is available from the Python Package Index via `pip install mlsynth`, with source and issue tracker at https://github.com/jgreathouse9/mlsynth and documentation at https://mlsynth.readthedocs.io.

The paper is reproducible from its own source. Rendering `paper/paper.qmd` re-runs every code block above and regenerates each number and figure from the installed library, against the package versions pinned in `paper/requirements.txt`. The handful of analyses that draw random numbers, the simulation-based scpi prediction intervals in the two cross-method

comparisons and the MAREX design search, fix a seed at the call site, so their output is stable from one render to the next.

The benchmark suite that holds each estimator to its outside answer is described in Section 5. Its reference cross-checks are frozen at pinned commits and run only where the other language's toolchain is present, skipping cleanly otherwise (for example in routine continuous integration, which carries no R), while the paper-number checks need nothing beyond `mlsynth` and run everywhere. The install scripts, the re-run harness, and the full replication code are provided as the replication materials accompanying this article and in the `benchmarks` suite of the source repository.

References

Abadie A (2021). "Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects." *Journal of Economic Literature*, **59**(2), 391–425. doi:10.1257/jel.20191450.

Abadie A, Diamond A, Hainmueller J (2010). "Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California's Tobacco Control Program." *Journal of the American Statistical Association*, **105**(490), 493–505. doi:10.1198/jasa.2009.ap08746.

Abadie A, Diamond A, Hainmueller J (2011). "**Synth**: An R Package for Synthetic Control Methods in Comparative Case Studies." *Journal of Statistical Software*, **42**(13), 1–17. doi:10.18637/jss.v042.i13.

Abadie A, Diamond A, Hainmueller J (2015). "Comparative Politics and the Synthetic Control Method." *American Journal of Political Science*, **59**(2), 495–510. doi:10.1111/ajps.12116.

Abadie A, Gardeazabal J (2003). "The Economic Costs of Conflict: A Case Study of the Basque Country." *American Economic Review*, **93**(1), 113–132. doi:10.1257/000282803321455188.

Abadie A, L'Hour J (2021). "A Penalized Synthetic Control Estimator for Disaggregated Data." *Journal of the American Statistical Association*, **116**(536), 1817–1834. doi:10.1080/01621459.2021.1971535.

Abadie A, Zhao J (2026). "Synthetic Controls for Experimental Design." 2108.02196, URL https://arxiv.org/abs/2108.02196.

Agarwal A, Shah D, Shen D (2026). "Synthetic Interventions: Extending Synthetic Controls to Multiple Treatments." *Operations Research*, **74**(2), 840–859. doi:10.1287/opre.2025.1590.

Amjad M, Shah D, Shen D (2018). "Robust Synthetic Control." *Journal of Machine Learning Research*, **19**(22), 1–51.

Arkhangelsky D, Athey S, Hirshberg DA, Imbens GW, Wager S (2021). "Synthetic Difference-in-Differences." *American Economic Review*, **111**(12), 4088–4118. doi:10.1257/aer.20190159.

Athey S, Bayati M, Doudchenko N, Imbens G, Khosravi K (2021). "Matrix Completion Methods for Causal Panel Data Models." *Journal of the American Statistical Association*, **116**(536), 1716–1730. doi:10.1080/01621459.2021.1891924.

Bai J (2009). "Panel Data Models with Interactive Fixed Effects." *Econometrica*, **77**(4), 1229–1279. doi:10.3982/ECTA6135.

Bayani M (2021). "Robust PCA Synthetic Control." 2108.12542, URL https://arxiv.org/abs/2108.12542.

Becker M, Klößner S (2018). "Fast and Reliable Computation of Generalized Synthetic Controls." *Econometrics and Statistics*, **5**, 1–19. doi:10.1016/j.ecosta.2017.08.002.

Ben-Michael E, Feller A, Rothstein J (2021). "The Augmented Synthetic Control Method." *Journal of the American Statistical Association*, **116**(536), 1789–1803. doi:10.1080/01621459.2021.1929245.

Brodersen KH, Gallusser F, Koehler J, Remy N, Scott SL (2015). "Inferring Causal Impact Using Bayesian Structural Time-Series Models." *The Annals of Applied Statistics*, **9**(1), 247–274. doi:10.1214/14-AOAS788.

Cattaneo MD, Feng Y, Palomba F, Titiunik R (2025). "**scpi**: Uncertainty Quantification for Synthetic Control Methods." *Journal of Statistical Software*, **113**(2), 1–38. doi:10.18637/jss.v113.i02.

Cattaneo MD, Feng Y, Titiunik R (2021). "Prediction Intervals for Synthetic Control Methods." *Journal of the American Statistical Association*, **116**(536), 1865–1880. doi:10.1080/01621459.2021.1979561.

Chernozhukov V, Wüthrich K, Zhu Y (2021). "An Exact and Robust Conformal Inference Method for Counterfactual and Synthetic Controls." *Journal of the American Statistical Association*, **116**(536), 1849–1864. doi:10.1080/01621459.2021.1920957.

Chernozhukov V, Wuthrich K, Zhu Y (2025). "Debiasing and t-tests for synthetic control inference on average causal effects." 1812.10820, URL https://arxiv.org/abs/1812.10820.

Colvin S, Pydantic Contributors (2024). ***pydantic**: Data Validation Using Python Type Hints*.

Di Stefano R, Mellace G (2024). "The inclusive Synthetic Control Method." 2403.17624, URL https://arxiv.org/abs/2403.17624.

Diamond S, Boyd S (2016). "**CVXPY**: A Python-Embedded Modeling Language for Convex Optimization." *Journal of Machine Learning Research*, **17**(83), 1–5.

Doudchenko N, Khosravi K, Pouget-Abadie J, Lahaie S, Lubin M, Mirrokni V, Spiess J, Imbens G (2021). "Synthetic Design: An Optimization Approach to Experimental Design with Synthetic Controls." *Advances in Neural Information Processing Systems*, **34**.

Dunford E (2021). "**tidysynth**: A Tidy Implementation of the Synthetic Control Method." https://github.com/edunford/tidysynth.

Engelbrektson O (2020). "**SyntheticControlMethods**: A Python Package for Causal Inference Using Synthetic Controls." https://github.com/OscarEngelbrektson/SyntheticControlMethods.

Fordham S (2022). "**pysyncon**: A Python Package for the Synthetic Control Method." https://github.com/sdfordham/pysyncon.

Hainmueller J (2012). "Entropy Balancing for Causal Effects: A Multivariate Reweighting Method to Produce Balanced Samples in Observational Studies." *Political Analysis*, **20**(1), 25–46. doi:10.1093/pan/mpr025.

Harris CR, Millman KJ, van der Walt SJ, *et al.* (2020). "Array Programming with NumPy." *Nature*, **585**, 357–362. doi:10.1038/s41586-020-2649-2.

Holland PW (1986). "Statistics and Causal Inference." *Journal of the American Statistical Association*, **81**(396), 945–960. doi:10.1080/01621459.1986.10478354.

Hsiao C, Ching HS, Wan SK (2012). "A Panel Data Approach for Program Evaluation: Measuring the Benefits of Political and Economic Integration of Hong Kong with Mainland China." *Journal of Applied Econometrics*, **27**(5), 705–740. doi:10.1002/jae.1230.

i Bastida JV (2022). "Predictor Selection for Synthetic Controls." 2203.11576, URL https://arxiv.org/abs/2203.11576.

Imbens GW, Rubin DB (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press, New York. doi:10.1017/CBO9781139025751.

Kellogg M, Mogstad M, Pouliot GA, Torgovitsky A (2021). "Combining Matching and Synthetic Control to Tradeoff Biases from Extrapolation and Interpolation." *Journal of the American Statistical Association*, **116**(536), 1804–1816. doi:10.1080/01621459.2021.1979562.

Lei J, G'Sell M, Rinaldo A, Tibshirani RJ, Wasserman L (2018). "Distribution-Free Predictive Inference for Regression." *Journal of the American Statistical Association*, **113**(523), 1094–1111. doi:10.1080/01621459.2017.1307116.

Lei L, Sudijono T (2025). "Inference for Synthetic Controls via Refined Placebo Tests." 2401.07152, URL https://arxiv.org/abs/2401.07152.

Liao C, Shi Z, Zheng Y (2026). "A Relaxation Approach to Synthetic Control." 2508.01793, URL https://arxiv.org/abs/2508.01793.

Liu J, Tchetgen Tchetgen E, Varjão C (2024). "Proximal Causal Inference for Synthetic Control with Surrogates." In S Dasgupta, S Mandt, Y Li (eds.), *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, pp. 730–738. PMLR. URL https://proceedings.mlr.press/v238/liu24a.html.

Malo P, Eskelinen J, Zhou X, Kuosmanen T (2024). "Computing Synthetic Controls Using Bilevel Optimization." *Computational Economics*, **64**(2), 1113–1136. doi:10.1007/s10614-023-10471-7.

Menchetti F, Bojinov I (2022). "Estimating the Effectiveness of Permanent Price Reductions for Competing Products Using Multivariate Bayesian Structural Time Series Models." *The Annals of Applied Statistics*, **16**(1), 414–435.

Meta Open Source (2024). "GeoLift: An End-to-End Geo-Experimentation Methodology." https://github.com/facebookincubator/GeoLift.

Park C, Tchetgen EJT (2025). "Single proxy synthetic control." *Journal of Causal Inference*, **13**(1), 20230079. doi:10.1515/jci-2023-0079.

Peng T, Agarwal A, Shah D (2022). "**causaltensor**: A Python Package for Causal Inference with Tensor/Matrix Completion." https://github.com/TianyiPeng/causaltensor.

Prakash S (2023). "Walmart Condensed Sales Data." Kaggle. URL https://www.kaggle.com/ds/3471234.

PyMC Labs (2024). "**CausalPy**: A Python Package for Causal Inference in Quasi-Experimental Settings." https://github.com/pymc-labs/CausalPy.

Qiu H, Shi X, Miao W, Dobriban E, Tchetgen Tchetgen E (2024). "Doubly robust proximal synthetic controls." *Biometrics*, **80**(2), ujae055. ISSN 1541-0420. doi:10.1093/biomtc/ujae055. https://academic.oup.com/biometrics/article-pdf/80/2/ujae055/58031293/ujae055.pdf, URL https://doi.org/10.1093/biomtc/ujae055.

Qiu J, Jammalamadaka SR, Ning N (2018). "Multivariate Bayesian Structural Time Series Model." *Journal of Machine Learning Research*, **19**(68), 1–33.

Robbins MW, Davenport S (2021). "**microsynth**: Synthetic Control Methods for Disaggregated and Micro-Level Data in R." *Journal of Statistical Software*, **97**(2), 1–31. doi:10.18637/jss.v097.i02.

Rubin DB (1974). "Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies." *Journal of Educational Psychology*, **66**(5), 688–701. doi:10.1037/h0037350.

Sevigny EL, Greathouse J, Medhin DN (2023). "Health, safety, and socioeconomic impacts of cannabis liberalization laws: An evidence and gap map." *Campbell Systematic Reviews*, **19**(4), e1362. doi:https://doi.org/10.1002/cl2.1362. https://onlinelibrary.wiley.com/doi/pdf/10.1002/cl2.1362, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/cl2.1362.

Shi X, Li KQ, Yu M, Miao W, Kuchibhotla AK, Hu M, Tchetgen ET (2026). "Theory for Identification and Inference with Synthetic Controls: A Proximal Causal Inference Framework." *Journal of the American Statistical Association*, **0**(0), 1–13. doi:10.1080/01621459.2026.2639734.

Shi Z, Huang J (2023). "Forward-Selected Panel Data Approach for Program Evaluation." *Journal of Econometrics*, **234**(2), 512–535. doi:10.1016/j.jeconom.2021.04.009.

Vega-Bayo A (2015). "An R Package for the Panel Approach Method for Program Evaluation: **pampe**." *The R Journal*, **7**(2), 105–121. doi:10.32614/RJ-2015-029.

Vovk V, Gammerman A, Shafer G (2005). *Algorithmic Learning in a Random World*. Springer, New York.

Xu Y (2017). "Generalized Synthetic Control Method: Causal Inference with Interactive Fixed Effects Models." *Political Analysis*, **25**(1), 57–76. doi:10.1017/pan.2016.2.

Xu Y, Zhou Q (2025). "Bayesian Synthetic Control with a Soft Simplex Constraint." *arXiv preprint arXiv:2505.00006*.

Yan G, Chen Q (2022). "**rcm**: A Command for the Regression Control Method." *The Stata Journal*, **22**(4), 842–883. doi:10.1177/1536867X221140960.

Yan G, Chen Q (2023). "**synth2**: Synthetic Control Method with Placebo Tests, Robustness Test, and Visualization." *The Stata Journal*, **23**(3), 597–624. doi:10.1177/1536867X231195278.

Zhang X, Wang W, Zhang X (2022). "Asymptotic Properties of the Synthetic Control Method." 2211.12095, URL https://arxiv.org/abs/2211.12095.

Affiliation:

Jared Greathouse
Department of Economics, Andrew Young School of Policy Studies
Georgia State University
Atlanta, GA, United States
E-mail: jgreathouse3@student.gsu.edu
URL: https://jgreathouse9.github.io/

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd